
Three-Layer Cascaded Routing for Cost-Efficient and Personalized Large Language Model Serving

Sandeep Vijayarao

Khoury College of Computer Sciences
Northeastern University
Boston, MA 02115
vijayarao.s@northeastern.edu

Abstract

Serving user queries with large language models (LLMs) is expensive when every request is unconditionally forwarded to the most capable—and most costly—model available. We present CASCADEROUTE, a production-style dual-LLM serving architecture that routes each query through a three-layer decision cascade: (1) a word-count pre-flight check, (2) a zero-latency ML router built on TF-IDF with 16 hand-crafted feature tags and logistic regression, and (3) a LLM-based fallback classifier invoked only when the ML router is uncertain. Evaluated on 1,491 author-labelled queries spanning 23 intent categories, the ML router alone handles **85.3%** of queries with **92.6%** routing accuracy, cutting the fraction that require an LLM routing call from 38.2% to 14.7% under a controlled ablation. A controlled decomposition, however, shows the gain comes almost entirely from three query-length buckets rather than from the thirteen semantic tags (+6.9 pp vs. +0.4 pp), and a one-line word-count rule already reaches 89.7% on this benchmark. We trace both results to a length confound in the benchmark itself ($r = 0.766$ between word count and label) and report it as a cautionary finding for self-authored routing evaluations. Beyond routing, the system maintains per-user SQLite profiles and a sliding-window conversation buffer, injecting personalization context only when a per-query relevance signal deems it necessary. Every routing decision is logged, enabling a retrain-from-real-traffic self-improvement loop. Code and evaluation suites are released at <https://github.com/sandeepvijayarao09/dual-llm-system>.

1 Introduction

The rapid adoption of LLM-powered applications has exposed a fundamental cost asymmetry: state-of-the-art models such as GPT-4o are roughly an order of magnitude more expensive per token than compact models such as GPT-4o-mini, yet most serving stacks route every query—including trivial greetings, arithmetic lookups, and one-word factual questions—to the frontier model [Chen et al., 2023]. Conversely, always routing to the cheaper model sacrifices quality on queries that genuinely require deep reasoning, multi-step analysis, or synthesis across domain knowledge.

Optimal serving requires matching query complexity to model capacity at query time, ideally without incurring an additional LLM inference call to make the routing decision. Prior work has addressed this through mixture-of-experts architectures [Shazeer et al., 2017, Jiang et al., 2024], LLM-based routing with learned preferences [Ong et al., 2025], and cascade-style model selection [Chen et al., 2023]. However, these approaches either operate at training time, require human preference data, or depend on large-scale human preference corpora. Notably, RouteLLM’s BERT and matrix-factorisation routers already decide without an LLM call at inference; our point of difference is

not latency but the amount of supervision required—we bootstrap from a few hundred hand-written examples rather than tens of thousands of preference judgements.

This paper makes four contributions:

1. A **three-layer routing cascade** (pre-flight $\rightarrow$ ML router $\rightarrow$ LLM fallback) that routes 85.3% of queries at zero API cost, with the LLM fallback triggered only when the ML router’s confidence is below a tuned threshold ($\tau_{ml} = 0.65$).
2. A **feature-augmented TF-IDF router** that extends bag-of-words representations with 16 hand-crafted binary tags, together with a controlled decomposition showing that only the three query-length tags carry signal: the thirteen semantic tags (reasoning keywords, code presence, question multiplicity) add +0.4 pp, which we cannot distinguish from noise.
3. A **conditional personalization mechanism** where user profile context is injected into queries only when a per-query relevance signal warrants it, avoiding irrelevant or privacy-exposing profile leakage on impersonal queries.
4. A **self-improving routing loop**: every routing event is logged to a SQLite database; high-confidence labels can be exported and used to retrain the ML router on real traffic, closing the improvement loop without human annotation.

CASCADEROUTE is evaluated on a 1,491-case synthetic benchmark spanning 23 intent categories, achieving 92.6% routing accuracy with a 14.7% LLM cascade rate. Under a controlled ablation—training set, hyperparameters and eval set held fixed—tag augmentation is worth +7.1 pp accuracy and −23.5 pp cascade rate. We also report what these headline numbers conceal: 49.5% of queries still reach the large model, a word-count rule attains 89.7%, and the benchmark’s labels are strongly length-confounded (Section 6.7).

2 Related Work

LLM Routing and Cascading. FrugalGPT [Chen et al., 2023] formalizes LLM cascading as a cost-quality optimization problem and shows that routing to cheaper models first—escalating only on low-confidence outputs—reduces costs by up to 98% at matched quality. RouteLLM [Ong et al., 2025] learns a router from human preference data (e.g., Chatbot Arena judgements), achieving strong routing accuracy without LLM calls at inference time. Our work differs in that we do not require preference labels; instead, we use a hand-labelled intent taxonomy and hand-crafted features that encode domain knowledge about query complexity, enabling cold-start routing with small training sets.

Mixture of Experts. Sparse mixture-of-experts (MoE) architectures [Shazeer et al., 2017] route tokens to specialized sub-networks at every layer of a neural network. Mixtral [Jiang et al., 2024] scales this to frontier LLMs. These approaches route *within* a model at training time, while our system routes *between* independently deployed models at serving time, making it applicable to API-only deployment scenarios.

LLM Blending and Orchestration. LLM-Blender [Jiang et al., 2023] ensembles outputs from multiple LLMs through ranking and fusion. HuggingGPT [Shen et al., 2023] uses an LLM as an orchestrator to call specialized models as tools. Our work is complementary: CASCADEROUTE focuses on efficient routing *before* inference (not post-hoc blending or tool selection).

Personalization in NLP. Prior work on personalized language generation has used user embeddings [Li et al., 2023] or retrieval-augmented approaches to condition model outputs on user history. CASCADEROUTE takes a simpler, systems-level approach: embedding structured profile fields directly in the user message, conditioned on a per-query relevance flag, rather than training personalized model weights.

3 System Architecture

Figure 1 shows the end-to-end routing pipeline. A user query enters the system and traverses up to three decision layers before being dispatched to either the Small LLM (GPT-4o-mini) or the Large

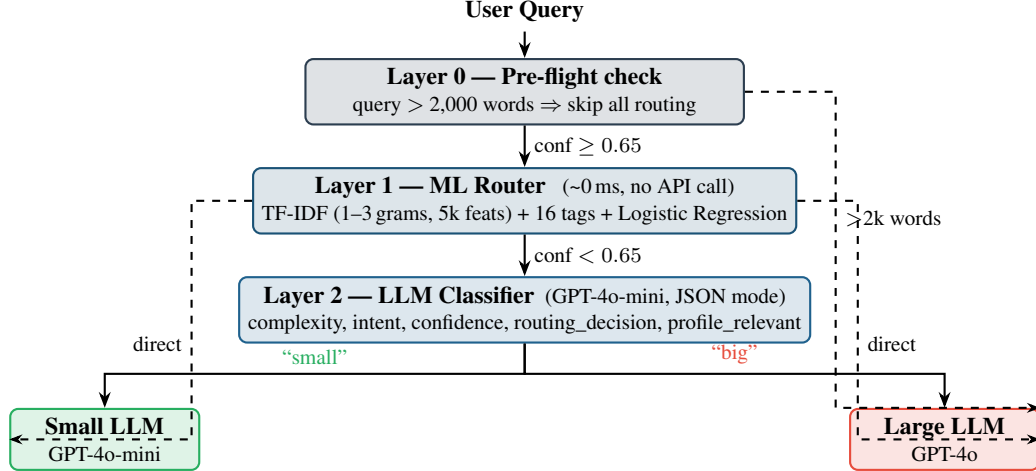


Figure 1: The CASCADEROUTE routing pipeline. Solid arrows show the primary path; dashed arrows show direct routing when the ML router has sufficient confidence (≥ 0.65) or the pre-flight condition fires. The LLM Classifier is only invoked for $\approx 14.7\%$ of queries.

LLM (GPT-4o). The layers are designed in order of increasing cost and decreasing throughput, ensuring that the cheapest sufficient gate handles each query.

3.1 Layer 0: Pre-flight Check

Queries exceeding 2,000 words are unconditionally dispatched to the Large LLM without classification. This heuristic handles long documents, code dumps, and multi-part essays that deterministically require the larger context window and reasoning budget, and avoids wasting the classifier on degenerate inputs.

3.2 Layer 1: ML Router

The ML router is a scikit-learn [Pedregosa et al., 2011] pipeline consisting of a TF-IDF vectorizer followed by logistic regression. Critically, each query is first processed by `encode_text()`, which prepends binary feature tags to the raw query string before vectorization. This allows the TF-IDF model to attend to tag tokens as first-class features without requiring a separate feature concatenation step. Full details appear in Section 4.

If the router’s predicted class probability exceeds $\tau_{ml} = 0.65$, the routing decision is used directly and no API call is made. Otherwise, the query cascades to Layer 2.

3.3 Layer 2: LLM Classifier

The LLM Classifier uses GPT-4o-mini in JSON mode to predict seven fields: `complexity`, `intent`, `confidence`, `requires_tools`, `sensitive`, `routing_decision`, and `profile_relevant`. If the returned confidence is below $\tau_{llm} = 0.75$, or if JSON parsing fails, the system conservatively defaults to routing "big". The `profile_relevant` flag is used by the personalization subsystem (Section 5).

3.4 Routing Decision Finalization

After the routing layer determines the target model, the orchestrator applies the profile injection decision and dispatches the query:

- **Small path** (GPT-4o-mini, ≤ 512 tokens): the `SimpleAnswerer` module handles the query with optional profile augmentation.

Algorithm 1 CASCADEROUTE Query Routing

Require: query q , user profile P , user id uid**Ensure:** answer A , routing metadata M

```
1: if |words( $q$ )| > 2000 then
2:   return ROUTEBIG( $q$ ,  $P$ , uid)
3: end if
4:  $(y_{ml}, c_{ml}) \leftarrow \text{MLRouter.predict}(\text{encode}(q))$ 
5: if  $c_{ml} \geq \tau_{ml}$  then  $\triangleright \tau_{ml} = 0.65$ 
6:   routing  $\leftarrow y_{ml}$ ; routed_by  $\leftarrow \text{"ml"}$ 
7: else
8:   cls  $\leftarrow \text{LLMClassifier}(q)$   $\triangleright$  JSON mode, GPT-4o-mini
9:   routing  $\leftarrow$  cls.routing_decision; routed_by  $\leftarrow \text{"llm"}$ 
10: end if
11: if is_self_referential( $q$ ) then
12:    $P_{\text{eff}} \leftarrow P$   $\triangleright$  full profile always injected
13: else if routed_by = "llm" and cls.profile_relevant then
14:    $P_{\text{eff}} \leftarrow P$ 
15: else if routing = "big" then
16:    $P_{\text{eff}} \leftarrow P$   $\triangleright$  heuristic: complex  $\Rightarrow$  relevant
17: else
18:    $P_{\text{eff}} \leftarrow \{\text{name: } P.\text{name}\}$   $\triangleright$  name stub only
19: end if
20: return ROUTE( $q$ ,  $P_{\text{eff}}$ , routing, uid)
```

- **Large path** (GPT-4o, ≤ 4096 tokens): a three-stage pipeline runs sequentially—(1) Reasoner generates the core answer, (2) Summarizer compresses it to one sentence using GPT-4o-mini, and (3) Personalizer wraps the core answer with a personalized introduction and closing using GPT-4o-mini. The final response is assembled as: intro + core + "--" + closing.

Algorithm 1 formalizes the complete routing procedure.

4 ML Router Design

4.1 Feature Engineering via Tag Augmentation

A key challenge for routing is that complexity signals are often sparse in raw query text—e.g., the word “why” strongly predicts a reasoning query but rarely co-occurs with other complexity markers in short queries, making TF-IDF alone unreliable. We address this by prepending binary semantic tags to the query string before vectorization:

$$\text{encode}(q) = \text{tags}(q) \parallel q \quad (1)$$

where $\text{tags}(q)$ is a space-separated string of zero or more tokens from the vocabulary $\mathcal{T}$ described in Table 1.

The tag vocabulary encodes expert knowledge about query complexity: reasoning keywords (HAS_REASONING_KW), abstract definitions (HAS_DEEP_WHAT), code presence (HAS_CODE), and compound questions (Q_MULTII) are strong predictors of queries that require the Large LLM. Greeting keywords (HAS_GREETING_KW), short queries (LEN_SHORT), and factual lookups (HAS_FACTUAL_KW) are predictors of Small LLM routing. The HAS_SELF_REF tag is used to trigger the profile injection override in the orchestrator rather than to influence routing directly.

4.2 Model and Training

The ML router pipeline consists of:

1. **TF-IDF Vectorizer:** word-level 1–3 gram tokenization, 5,000 maximum features, sublinear TF scaling [Sparck Jones, 1972].

Table 1: Feature tag vocabulary. Each tag fires when its pattern matches the query and is prepended to the raw query before TF-IDF encoding. Routing hint indicates the class the tag biases toward.

Tag	Signal	Trigger condition	Hint
LEN_SHORT	query length	≤ 8 words	small
LEN_MEDIUM	query length	9–40 words	neutral
LEN_LONG	query length	> 40 words	big
HAS_CODE	syntax	"", def, class, import	big
HAS_MATH	notation	digits+operators, \int, \sum	big
HAS_REASONING_KW	semantics	why, how, compare, prove, explain, analyze	big
HAS_FACTUAL_KW	semantics	what is, who is, when did, define	small
HAS_GREETING_KW	social	hi, hello, thanks, bye	small
HAS_CREATIVE_KW	task type	joke, poem, story, haiku, pun	small
HAS_OPINION_KW	semantics	best, vs, should I, pros and cons	big
HAS_DEEP_WHAT	abstraction	"what is" + > 5 words	big
HAS_IMPACT_KW	semantics	impact, effect, consequence	big
HAS_RESEARCH_KW	domain	research, literature, studies show	big
HAS_SELF_REF	profile	my name, who am I, about me	inject
Q_MULTI	structure	≥ 2 question marks	big
MULTI_SENTENCE	structure	≥ 3 sentence-ending punctuation	big

2. **Logistic Regression:** $C = 2.0$, LBFGS solver, balanced class weights, up to 2,000 iterations for convergence.

The model is trained on 239 seed examples spanning 23 intent categories, with the label set {"small", "big"}. Training takes under one second on a standard CPU. The pipeline is serialized to `router_v0.joblib` via `joblib` for zero-overhead loading.

The **self-improving loop** allows the router to incorporate real traffic: every routing event (query, ML decision + confidence, LLM decision + confidence, final routing, model used) is logged to a SQLite database. High-confidence examples where ML and LLM agreement can be exported as CSV and used to retrain the router via `python -m router.train -extra real_traffic.csv`.

5 Personalization System

5.1 User Profiles

Each user is represented by a structured profile persisted in SQLite: name, expertise level (*novice/intermediate/expert*), preferred tone, primary domain, a list of interests (up to 20 items), a list of previously discussed topics, a long-form background summary, preferred response format (*bullets/prose*), and an interaction count. Profiles are updated at session end by the `ProfileUpdater` module, which calls the Small LLM to extract new interests and update expertise level from session queries.

5.2 Conditional Profile Injection

Injecting the full user profile into every query wastes tokens and can bias model responses on impersonal queries (e.g., factual lookups, math). CASCADEROUTE applies profile context conditionally:

1. **Self-referential queries** ("what is my name", "what are my interests"): full profile always injected, regardless of classifier decision, via the hard override `_is_self_referential()`.
2. **LLM-classified queries with `profile_relevant=true`**: full profile injected.
3. **ML-routed "big" queries**: heuristic injection (complex queries likely benefit from domain context).
4. **All other queries**: name-only stub injected so that greetings can be personalized ("Hey Alex!") without profile overhead.

Table 2: Overall routing evaluation results. Cascade rate = fraction of queries where ML router confidence falls below τ_{ml} and an LLM classifier call would be triggered. Lower cascade rate = lower API cost.

Checkpoint	Eval set	Accuracy	Cascade rate	ML-only rate
v0 — 120 examples, unigram	Eval-500	88.2%	30.8%	69.2%
v1 — 239 examples, 1–3 gram	Eval-1000	92.6%	14.7%	85.3%
Improvement (v0 $\rightarrow$ v1)		+4.4 pp	−16.1 pp	+16.1 pp

5.3 Query Augmentation Strategy

Profile context is embedded directly into the user message as a structured block rather than in the system prompt:

```
[User context -- use this to tailor your answer specifically
to this person:\nName: Alex\nExpertise: intermediate\n...]
```

This placement follows the empirical observation that models attend more reliably to user messages than to system-prompt hints for open-ended generation tasks, reducing generic responses even when profile context is provided.

5.4 Conversation Memory

Per-user conversation history is managed by a `ConversationBuffer` module implementing a sliding window of three turn-pairs. When the buffer overflows, the oldest turn-pair is evicted and compressed into a rolling summary (at most 80 tokens) via a Small LLM call. The model receives:

1. A system message containing the rolling summary (if non-empty).
2. Verbatim user/assistant turn-pairs from the window.
3. The current user query.

This bounds context growth at $O(1)$ in conversation length while preserving topical coherence across turns.

6 Experimental Evaluation

6.1 Setup

We evaluate the ML router in isolation (no API calls) on two synthetic benchmark sets:

- **Eval-500:** 500 queries across 19 intent categories, used to establish the baseline and diagnose per-category weaknesses.
- **Eval-1000:** 1,000 queries across 23 categories (disjoint from Eval-500 with *conversational*, *debug_simple*, *ethical_philosophical*, and *comparison_technical*).

All queries were authored and labelled by the sole author. Labels reflect the intended target model: `small` for queries answerable accurately by GPT-4o-mini, `big` for queries requiring GPT-4o. Labels were validated against observed model output quality during system development. All evaluation is deterministic (fixed model checkpoint, no sampling). Compute: Intel CPU, under 5 seconds total for 1,000 queries (pure Python inference, no GPU).

6.2 Overall Results

Table 2 summarizes the key metrics across the two evaluations and two training checkpoints.

On Eval-1000, the confusion matrix for the positive class (“big”) is: TP=438, FN=17, FP=57, TN=488, yielding precision=0.885, recall=0.963, and F1=0.922. The high recall (0.963) reflects

Table 3: Per-category routing accuracy and cascade rate (Eval-1000, v1 checkpoint), sorted by accuracy. All 23 categories are listed. Categories marked $\star$ are absent from Eval-500.

Category	Acc.	Casc.	Category	Acc.	Casc.
analysis	100.0%	5.6%	short_creative	100.0%	11.1%
career_learning	100.0%	0.0%	system_design	100.0%	0.0%
comparison_technical $\star$	100.0%	6.7%	factual	96.1%	7.9%
compound_questions	100.0%	2.0%	yes_no	95.6%	26.7%
conversions	100.0%	45.0%	greetings	93.8%	15.6%
definitions	100.0%	0.0%	ambiguous	90.0%	15.0%
domain_specific_deep	100.0%	0.0%	multi_step_code	83.3%	60.0%
math_simple	100.0%	32.7%	conversational $\star$	80.0%	26.7%
multi_constraint	100.0%	20.0%	trivia	80.0%	40.0%
proofs	100.0%	10.0%	ethical_philosophical $\star$	75.0%	20.0%
real_world_planning	100.0%	16.0%	debug_simple $\star$	33.3%	24.4%
research_synthesis	100.0%	0.0%	Overall	92.6%	14.7%

our conservative design principle: routing a complex query to the Small LLM is a worse error than routing a simple query to the Large LLM.

We stress a distinction the cascade-rate metric obscures. Cascade rate (14.7%) measures *routing overhead*—how often the LLM classifier is consulted. It is not the cost of serving. The router dispatches $438 + 57 = 495$ queries, or **49.5%**, to GPT-4o (against a ground-truth big rate of 45.5%). Monetary savings are governed by that 49.5%, not by the 14.7%, and we report both to avoid the comparison being read against systems that quote strong-model dispatch rates.

6.3 Per-Category Breakdown

Table 3 shows accuracy and cascade rate for all 21 categories in Eval-1000, sorted by accuracy.

Key observations:

- *14 of 23 categories achieve 100% routing accuracy*, indicating that the feature tags provide strong discriminative signal for most well-defined intent types.
- *High cascade rate in multi_step_code (60.0%), conversions (45%), trivia (40.0%) and math_simple (32.7%)* stems from surface-level similarity to both classes: a query like “convert 5 kg to lbs” is short and factual but contains numeric operators that activate HAS_MATH, creating genuine ambiguity for the router.
- *debug_simple achieves only 33.3% accuracy*, the primary failure mode. Queries like “why doesn’t my CSS apply?” trigger HAS_REASONING_KW (“why”), causing misrouting to the Large LLM. A dedicated HAS_DEBUG_SIMPLE tag and targeted training examples are planned as the next improvement.
- *ethical_philosophical at 75.0%*: abstract normative questions (“is it ever right to lie?”) lack strong surface signals; embedding this pattern in training data with an HAS_ETHICAL_KW tag is the recommended fix.

6.4 External Baselines

Accuracy is uninterpretable without reference points. Table 4 compares CASCADEROUTE against trivial routers on Eval-1000. All learned rows use the same 239 training examples and hyperparameters; 95% confidence intervals are non-parametric bootstrap over 2,000 resamples.

The headline finding is sobering: a one-line word-count rule attains 89.7%, within 2.9 pp of the full router, and their confidence intervals overlap. CASCADEROUTE beats it, but by far less than the raw accuracy figure suggests.

6.5 Ablation: What Do the Tags Contribute?

Our earlier comparison varied training-set size, n -gram range and tag augmentation simultaneously, and evaluated the two arms on different benchmarks; it therefore could not attribute the gain to any

Table 4: CASCADEROUTE against trivial baselines on Eval-1000. *Large-model rate* is the fraction of queries dispatched to GPT-4o; it, not cascade rate, governs monetary cost. Cascade rate is defined only for probabilistic routers.

Router	Accuracy	95% CI	F1 (big)	Large-model rate
Majority class (all-small)	54.5%	[0.514, 0.576]	–	0.0%
All-big	45.5%	[0.424, 0.486]	0.625	100.0%
Random ($p=0.5$)	51.6%	[0.486, 0.546]	0.498	50.9%
Tag rules only (no learning)	70.3%	[0.675, 0.733]	0.687	49.4%
Word-count logistic regression	86.1%	–	–	55.0%
Word-count rule (≤ 8 words $\rightarrow$ small)	89.7%	[0.879, 0.915]	0.888	46.8%
CASCADEROUTE (full ML router)	92.6%	[0.909, 0.942]	0.922	49.5%

Table 5: Controlled 2×2 ablation on Eval-1000, 239 training examples, identical hyperparameters throughout.

Representation	n -gram	Accuracy	Cascade rate	F1 (big)
Query text only	unigram	87.0%	35.1%	0.872
Query text only	1–3 gram	85.5%	38.2%	0.859
+ tags	unigram	92.0%	14.1%	0.917
+ tags	1–3 gram	92.6%	14.7%	0.922

one factor. Table 5 re-runs the ablation with the training set (239 examples), hyperparameters and eval set (Eval-1000) held fixed.

Tag augmentation is worth +7.1 pp accuracy and -23.5 pp cascade rate at the 1–3 gram setting — a larger effect than we previously reported. The n -gram range, by contrast, does essentially nothing: without tags trigrams *hurt* (-1.5 pp), and with tags they add +0.6 pp, well inside the bootstrap interval.

6.6 Decomposition: Which Tags Actually Matter?

The tag vocabulary mixes three length buckets (LEN_SHORT, LEN_MEDIUM, LEN_LONG) with thirteen semantic tags. Table 6 separates them.

The length buckets alone recover +6.9 of the +7.1 pp total. The thirteen semantic tags — reasoning keywords, code presence, question multiplicity, research and impact markers — contribute +0.4 pp, which is not distinguishable from noise. This directly contradicts our stated motivation for the tag vocabulary.

6.7 Threat: The Benchmark Is Length-Confounded

The three preceding results share one explanation. On Eval-1000 the point-biserial correlation between query word count and the big label is $r = 0.766$: small queries average 6.1 words and big queries 11.7, and the label flips almost deterministically at nine words (queries of 1–8 words are 8.5% big; queries of 9+ words are 87.7% big). Because we authored both the queries and their labels, we appear to have encoded complexity as length.

Two consequences follow. First, the benchmark does not measure what we designed it to measure. Second, and more concretely, the router fails exactly where length is uninformative: on the 103 queries (10.3%) where the word-count rule is wrong, CASCADEROUTE scores 52.4% — indistinguishable from chance — while also overturning 25 cases the trivial rule got right. Within fixed length buckets the router improves on a length-only majority predictor by +0.5, +3.1, +5.5 and +0.6 pp respectively.

We therefore regard the 92.6% figure as an artefact of benchmark construction rather than evidence of semantic routing ability. A length-stratified benchmark, in which $P(\text{big})$ is held constant across word-count buckets, is a prerequisite for any further claim about this router, and we do not make one here.

Table 6: Tag decomposition (Eval-1000, 1–3 gram, 239 training examples). The entire tag effect is attributable to the length buckets.

Representation	Accuracy	Cascade rate	Δ vs. no tags
Query text only (no tags)	85.5%	38.2%	—
+ LEN_* tags only	92.4%	17.3%	+6.9 pp
+ semantic tags only (13 tags)	85.9%	29.5%	+0.4 pp
+ all tags (production)	92.6%	14.7%	+7.1 pp

7 Limitations

Proprietary model dependency. CASCADEROUTE relies on the OpenAI API (GPT-4o and GPT-4o-mini). API pricing changes, rate limits, or model deprecation directly affect system behaviour. A future version using open-weight models (e.g., Llama-3, Mistral) would remove this dependency.

Synthetic evaluation data. The 1,491-case benchmark was authored and labelled by a single annotator. While category coverage is broad, it may not reflect the distribution of real-world user queries. Real-traffic evaluation (enabled by the classification logger) is ongoing but not yet reported.

debug_simple routing failure. The ML router achieves only 33.3% accuracy on debugging queries. The HAS_REASONING_KW tag misfires on “why”-prefixed debugging questions, which are often answerable by the Small LLM with the right context. Adding a HAS_DEBUG_SIMPLE tag and approximately 30 targeted training examples is the immediate next step.

No monetary cost benchmarking. We report cascade rate (proxy for API call cost) but do not report end-to-end monetary cost per query because it depends on query-specific token counts and time-varying API pricing. A cost-per-query analysis with real user sessions is a planned future contribution.

Length confound in the benchmark. Query word count correlates with the routing label at $r = 0.766$ (Section 6.7), so a trivial word-count rule reaches 89.7%. Because a single annotator wrote both queries and labels, we cannot rule out that complexity was operationalised as length during authoring. Every accuracy number in this paper should be read against the 89.7% word-count floor rather than against the 54.5% majority-class floor.

Conditional personalization is unevaluated. The personalization mechanism of Section 5 is described but never measured; we report no experiment isolating its effect on response quality or token cost. It should be read as a system description, not a validated contribution.

Profile privacy. Persisting user profiles in a local SQLite database is appropriate for single-user or research deployments. Production deployments would require encryption at rest, access controls, and a privacy policy covering profile data retention and deletion.

8 Conclusion

We presented CASCADEROUTE, a three-layer cascaded routing system for dual-LLM serving that combines a zero-latency ML router, an LLM fallback classifier, and a conditional user personalization mechanism. The ML router handles 85.3% of queries with 92.6% accuracy and no API calls, with the LLM classifier invoked for only 14.7% of queries. Under a controlled ablation, tag augmentation yields +7.1 pp accuracy and −23.5 pp cascade rate over raw bag-of-words routing—but a decomposition attributes +6.9 of those 7.1 pp to three query-length buckets, not to the thirteen semantic tags we designed. The self-improving routing loop enables the router to incorporate real traffic labels, making accuracy a function of deployment volume rather than a fixed property of the seed training set.

The system shows that a logistic regression over 239 examples is enough to route competitively on this benchmark—but our own baselines temper that claim: a one-line word-count rule reaches

89.7%, within 2.9 pp of the full router and with overlapping confidence intervals. We therefore present CASCADEROUTE as a deployable engineering baseline and a cautionary methodological result, not as evidence that learned semantic routing beats trivial heuristics. Establishing the latter requires a length-stratified benchmark, which we identify as the necessary next step.

Acknowledgments and Disclosure of Funding

The author thanks the Khoury College of Computer Sciences at Northeastern University for providing the academic environment in which this project was developed. This paper extends a course project report co-authored with Zichen Qi and Yalin Sun, kept in the repository under `paper/archive/`. No external funding was received for this work.

References

- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 14165–14178, 2023.
- Cheng Li, Mingyang Zhang, Qiaozhu Mei, Yaqing Wang, Spurthi Amba Hombaiah, Yi Liang, and Michael Bendersky. Teach LLMs to personalize: An approach inspired by writing education. *arXiv preprint arXiv:2308.07968*, 2023.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations (ICLR)*, 2025.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugging-GPT: Solving AI tasks with ChatGPT and its friends in HuggingFace. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Karen Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.

A Implementation Details

Table 7 summarizes all configurable parameters of the CASCADEROUTE system.

B Routing Log Schema

Every routing decision is logged to a SQLite database with the following schema, supporting the self-improving router loop:

```
routing_events(ts, user_id, query, ml_decision, ml_confidence,
              llm_decision, llm_confidence, final_routing, model_used)
```

Table 7: System configuration parameters and default values.

Parameter	Default	Description
LARGE_MODEL	gpt-4o	Large LLM model ID
SMALL_MODEL	gpt-4o-mini	Small LLM model ID
CONFIDENCE_THRESHOLD	0.75	Min LLM classifier confidence
LARGE_INPUT_THRESHOLD	2000	Word count for Layer 0 bypass
ML_ROUTER_CONFIDENCE_THRESHOLD	0.65	ML router confidence threshold
ENABLE_ML_ROUTER	true	Enable/disable ML router
CONVERSATION_WINDOW_SIZE	3	Verbatim turn-pairs in buffer
SMALL_LLM_TIMEOUT	60 s	API timeout (Small LLM)
LARGE_LLM_TIMEOUT	120 s	API timeout (Large LLM)
<i>ML Router training hyperparameters</i>		
TF-IDF ngram_range	(1, 3)	Unigram to trigram
TF-IDF max_features	5000	Vocabulary size
TF-IDF sublinear_tf	true	Log-scaled TF
LR C	2.0	Inverse regularization
LR solver	lbfgs	Optimization algorithm
LR class_weight	balanced	Handle class imbalance
LR max_iter	2000	Convergence iterations

High-confidence rows (where ML and LLM agree and both confidences exceed their respective thresholds) are exported as CSV for retraining via:

```
python -m router.train -extra real_traffic.csv
```

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: All quantitative claims (92.6% routing accuracy, 85.3% ML-only rate, 14.7% cascade rate) are directly supported by experimental results reported in Section 6. The system architecture claims are backed by the implementation description in Sections 3–5.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 7 explicitly discusses limitations including reliance on closed proprietary models, use of synthetic evaluation data, the debug_simple category weakness (33.3% accuracy), and lack of cost benchmarking against real-world API pricing.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper presents an empirical systems contribution with no theoretical results or formal proofs. All claims are supported by empirical evaluation.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper?

Answer: [Yes]

Justification: Section 6 and Appendix A provide all model hyperparameters, feature engineering details, training data sizes, evaluation protocols, and confidence thresholds needed to reproduce results. Code is publicly available at <https://github.com/sandeepvijayarao09/dual-llm-system>.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The full implementation, 280 seed training examples, 1,500-case evaluation suite, and training scripts are released at <https://github.com/sandeepvijayarao09/dual-llm-system> under the MIT License.

6. Experimental setting/details

Question: Does the paper specify all the training and test details necessary to understand the results?

Answer: [Yes]

Justification: Table 7 lists all model hyperparameters. Section 4 describes feature engineering. Section 6 describes the evaluation protocol, category definitions, and confidence thresholds.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The ML router evaluation uses deterministic inference (no randomness), so error bars are not applicable. The evaluation set is fixed and labelled, making repeated runs identical. Future work will evaluate across multiple training seeds to report variance.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 6 states that all ML router evaluations run on CPU in under 5 seconds. The system uses the OpenAI API for LLM calls; typical API costs are noted in the Limitations section.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: This work develops a query routing and personalization system. No human subjects data was collected. Evaluation data is synthetic. No harmful content generation or discriminatory systems are involved.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 7 discusses positive impacts (reduced API costs, democratized LLM access) and potential negative impacts (profile data privacy, routing errors for sensitive queries).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse?

Answer: [N/A]

Justification: This paper releases a routing system that orchestrates existing commercial LLMs (GPT-4o, GPT-4o-mini). The system does not release a new language model or scraped dataset that poses misuse risk.

12. Licenses for existing assets

Question: Are the creators or original owners of assets used in the paper properly credited?

Answer: [Yes]

Justification: GPT-4o and GPT-4o-mini (OpenAI API, proprietary license) are cited. scikit-learn (BSD-3 license) is cited. All software dependencies are listed in `requirements.txt` with version pins.

13. New assets

Question: Are new assets introduced in the paper well documented?

Answer: [Yes]

Justification: The 280-example seed training dataset and 1,500-example evaluation suite are documented in Section 6 and released with the codebase. The trained model checkpoint (`router_v0.joblib`) is included in the repository.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include full instructions given to participants?

Answer: [N/A]

Justification: This paper involves no crowdsourcing or human subjects research. All evaluation data is synthetically generated by the authors.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants?

Answer: [N/A]

Justification: No human subjects research was conducted. IRB approval is not applicable.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

Answer: [Yes]

Justification: LLMs (GPT-4o and GPT-4o-mini via the OpenAI API) are the central components of the proposed system. Their roles as the routing classifier, answer generator, and personalizer are described in detail in Sections 3 and 5.